

Publishing on CMKS

An architecture overview for developers who want to get an application running in a CMKS cluster — what the platform is, who owns which part, and what you have to build, configure and *ask for*.

Platform Gardener Kubernetes on Cleura (OpenStack) **Delivery** ArgoCD, GitOps
Source of truth CMKS - Infra (Terraform)

CONTENTS

- | | |
|-----------------------|-----------------------------|
| 01 Three rules | 09 Outbound traffic |
| 02 The platform | 10 Capios systems & the VPN |
| 03 Who owns what | 11 Secrets |
| 04 How deploys happen | 12 Platform services |
| 05 Getting onboarded | 13 Guardrails |
| 06 Your manifests | 14 Life without kubectl |
| 07 Deployment styles | 15 Checklist |
| 08 Inbound traffic | 16 Who to ask |

Three rules

Everything else on this page follows from these. If you internalise nothing else, internalise these and [section 09](#).

RULE 1

You will never get a kubeconfig.

No `kubect1` for developers — not in dev, not in test, not in prod. This is deliberate and is not granted per-app. Everything you'd do with `kubect1`, you do in ArgoCD and Grafana instead — reached over the Capiro VPN via the Jumphost.

RULE 2

Git is the only way in.

You don't deploy — you commit. ArgoCD watches your **deployment repo** and reconciles the cluster to match it. Anything applied or edited by hand is reverted automatically.

RULE 3

Network changes belong to other teams.

Hostnames, TLS, egress rules and — above all — connectivity to Capiro internal systems are changes owned by others, with their own lead times. Plan for them at design time.

The platform

CMKS is a set of Gardener-managed Kubernetes clusters running at **Cleura**, a European OpenStack provider. The entire platform — clusters, operators, tenants, applications, policy — is declared as Terraform in the **CMKS - Infra** repository. Nothing is clicked into existence.

Around the clusters sit three external dependencies you'll interact with: **Azure** (DNS, Key Vault for secrets, Entra ID for SSO), **Azure DevOps** (your Git repos, pipelines and container registry), and the **Cleura ↔ Capiro VPN service** that carries any traffic to Capiro internal systems.

Clusters and environments

CLUSTER	ENVIRONMENT	INTERNAL HOSTNAMES	EXTERNAL HOSTNAMES
dev02	dev	{service}.dev02.capiro.eu	{service}.capio.eu
test02	test	{service}.test02.capiro.eu	{service}.test.capiro.eu
prod01	prod	{service}.prod01.capiro.eu	{service}.prod.capiro.eu

Each environment designates one cluster as its **homeCluster**, which runs the shared platform services — ArgoCD, PostgreSQL and RabbitMQ. Your application runs on whichever clusters it is assigned, normally one per environment.

Who owns what

This is the most useful thing to internalise: **three repositories, three jobs**. Your source code and your deployment manifests do *not* live in the same place.

1 · Your source repo

you own it – anywhere, any layout

- `src/`, `tests/`, `Dockerfile`
- Your build pipeline
- Pushes the image to your ACR
- **ArgoCD never reads this repo**

2 · Deployment repo

you own it – one project, one repo per env

- Project `CMKS-<Tenant>-<App>`
- One repo per environment: `DEV`, `TEST`, `PROD`
- `deployment/` – manifests, chart or overlay
- **This is what ArgoCD syncs**

3 · CMKS - Infra

platform team owns it – request a change

- Namespace, ResourceQuota, LimitRange
- ArgoCD project + bootstrap Application
- Gateway listener and TLS certificate
- Key Vault wiring, `SecretStore`
- Your app's ACR and CMKS project
- Databases, queues, Redis
- NetworkPolicies – ingress and egress

Why the split matters

- **Building and deploying are separate acts.** A green build changes nothing in any cluster. A release happens when you change the image tag in a deployment repo.
- **One CMKS project per app, one repo per environment.** Your app gets a project `CMKS-<Tenant>-<App>` holding a `DEV`, `TEST` and `PROD` repo. Promoting dev → test → prod means applying the same change to the next environment's repo – not merging a branch inside one repo. Environments cannot drift by accident, and a prod release is always a deliberate, reviewable commit.
- **Your source repo is yours.** Language, layout, branching model, monorepo or not – the platform has no opinion and never reads it.
- **Only allowlisted repos work as an ArgoCD source.** The allowlist is configured by the platform team; anything else is rejected at sync.

RULE OF THUMB

If it is cluster-scoped, shared between tenants, or costs money, the platform team owns it.
If it lives inside your namespace and affects only your app, you own it.

How a deployment actually happens

Two independent pipelines. The first produces an image; the second releases it.

BUILD — NOTHING REACHES A CLUSTER

STEP 1

You push source

A commit lands in your own repo, wherever it lives.

STEP 2

Pipeline builds

Tests run, the image is pushed to your ACR. No cluster is touched.

RELEASE — THIS IS THE DEPLOYMENT

STEP 3

You bump the tag

Commit `image: myapp:1.4.2` to the deployment repo for that environment.

STEP 4

ArgoCD reconciles

It renders `deployment/` and applies the difference into your namespace.

STEP 5

Cluster matches Git

Continuously. Anything that drifts is pruned or reverted.

DO NOT USE `:LATEST`

A mutable tag defeats the model. ArgoCD sees no change in Git, so nothing syncs — and `selfHeal` will not pull a new image for you. Use an immutable tag or a digest.

Your ArgoCD `Application` is created once by Terraform, named `<tenant>-<app>`, with this sync policy:

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

Two consequences worth spelling out:

- `prune: true` — delete a manifest from the deployment repo and the object is deleted from the cluster. There is no orphan state.
- `selfHeal: true` — any out-of-band change to a live object is reverted within minutes. Hotfixing in the cluster does not work, by design.

After bootstrap, Terraform never touches the Application again, so tenant admins can adjust it in the ArgoCD UI within their project's RBAC boundary.

Getting onboarded

You need a **tenant** (an organisational unit — `capioonline`, `capioinkop`, `capioDA` ...) and an **app** under it. You do not configure any of it yourself — you order it.

ORDER A TENANT OR AN APP

CMKS onboarding form →

Both a new tenant and a new app go through this same form. Have the answers below ready before you open it — it is the only route in, and a request that arrives some other way will be pointed back here.

What the form asks for

WHAT THEY NEED	EXAMPLE
App name and tenant	<code>bookingapi</code> , tenant <code>mytenant</code>
Cost centre (KST) and legal entity	<code>1234</code> , <code>capio-sverige-ab</code>
Environments the app will run in	<code>dev</code> , <code>test</code> , <code>prod</code> — or a subset to start
Hostnames, internal or external	<code>bookingapi.prod01.capio.eu</code> · <code>internal</code>
Deployment style	plain manifests, Helm or Kustomize — see section 07
Entra ID groups for admins / developers	object IDs — these become your ArgoCD access
Databases, queues or Redis needed	see section 12
Every outbound destination	host, IP, port, protocol — see 09 and 10
Resource footprint	CPU, memory, pod count — becomes your quota

What you get back

The platform team provisions all of this and hands you the names you need:

YOU RECEIVE	SHAPE
A namespace	<code>ns-<tenant>-<app></code> , with quota, limits and network policies applied
An Azure Container Registry for the app	Created together with your config — this is where your build pushes
A CMKS project in Azure DevOps	<code>CMKS-<Tenant>-<App></code> , holding one deployment repo per environment
An ArgoCD Application per environment	Named <code><tenant>-<app></code> , already pointed at the right repo and namespace
A Gateway listener and TLS certificate	One per hostname — issued and renewed automatically
A Key Vault and a <code>SecretStore</code>	Per environment, wired up and ready to pull from
Credentials for platform services	As Secrets in your namespace

Azure DevOps organisation: capiomks

```
└─ CMKS-<Tenant>-<App>/      ← one project per app
   └─ DEV                     ← one repo per environment
   └─ TEST
   └─ PROD
```

Your source repository is separate and stays wherever your team already keeps it.

YOUR IMAGE GOES TO YOUR APP'S ACR

That registry is provisioned along with your app config, and your build pipeline pushes there. Registry restrictions are enforced by policy — build against the ACR you were given from the start, rather than discovering the constraint later.

Changing things later

Quotas, hostnames, egress rules, databases and queues are all platform-team changes — raise a request the same way. What you can change freely, without asking anyone, is anything inside your deployment repo.

What you write in the deployment repo

■ YOU OWN THIS

Everything below lives in `deployment/` and is synced by ArgoCD:

```
deployment/
├─ deployment.yaml      # your workload
├─ service.yaml         # ClusterIP only
├─ httproute.yaml       # only if you have a hostname
└─ externalsecret.yaml  # only if you need secrets
```

The parts that get first deployments rejected

```
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: bookingapi # NetworkPolicies select on this
    spec:
      automountServiceAccountToken: false # enforced by Kyverno
      securityContext:
        seccompProfile:
          type: RuntimeDefault # ENFORCED – rejected without it
      containers:
        - name: api
          image: cmksprodbookingapi-....azurecr.io/api:1.4.2
          securityContext:
            readOnlyRootFilesystem: true # ENFORCED – rejected without it
            allowPrivilegeEscalation: false
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
          volumeMounts:
            - { name: tmp, mountPath: /tmp } # writable scratch
      volumes:
        - { name: tmp, emptyDir: {} }
```

MOST COMMON FIRST FAILURE

`readOnlyRootFilesystem: true`. If your runtime writes to disk — temp files, caches, .NET data-protection keys, nginx pid files — mount an `emptyDir` at each of those paths. The pod will not start otherwise.

Deployment styles

ArgoCD detects what a `path` contains and renders it accordingly. You choose the style per app, and you can change it later by editing the Application in the ArgoCD UI — the platform does not force one.

STYLE	DETECTED BY	USE IT WHEN
Plain manifests	neither of the below	The default. One app, few resources, values differ little between environments.
Helm	<code>Chart.yaml</code> in the path	One templated chart with per-environment values, or a third-party chart.
Kustomize	<code>kustomization.yaml</code>	A shared base with small per-environment patches, without Helm templating.
App-of-apps	an Application rendering <code>Application</code> resources	One "app" is really several pieces you want to sync separately.
ApplicationSet	an <code>ApplicationSet</code> resource	Generating many similar Applications from a list or from Git.

Plain manifests

What every app on the platform uses today. ArgoCD applies every YAML in the path, recursively.

Because there is a **separate deployment repo per environment**, you do not need templating just to vary a hostname or an image tag — the files already differ per environment. Start here. Move to Helm or Kustomize only when duplication actually hurts.

Helm

Put a chart in the deployment repo and ArgoCD renders it with `helm template`. It uses `values.yaml` by default; to use another file or set values inline, edit the Application's `source.helm` block:

```
source:
  repoURL: git@ssh.dev.azure.com:v3/capiomks/CMKS-MyTenant-MyApp/PROD
  path: deployment
  targetRevision: main
  helm:
    valueFiles:
      - values.yaml
      - values-prod.yaml
    parameters:
      - name: image.tag
        value: "1.4.2"
```

PREFER A COMMITTED VALUES FILE

A value set inline in the Application is invisible to anyone reading the repo, and it is exactly the state that gets lost when an Application is recreated.

Community charts usually need trimming. Most bundle things tenants may not create:

CHART SHIPS	RESULT
CRDs	Rejected — all cluster-scoped resources are blocked
ClusterRole / ClusterRoleBinding	Rejected — same reason
Service of type LoadBalancer	Rejected by Kyverno
Ingress	Useless here — the platform uses Gateway API; write an HTTPRoute
No seccomp / writable rootfs	Pod rejected at admission

Charts with a `crds/` directory or an `rbac.create: true` default are the usual offenders. Check `helm template` output against the [guardrails](#) before adopting a chart. Rendering runs with a **300-second repo-server timeout**.

Kustomize

A `kustomization.yaml` in the path switches ArgoCD to `kustomize build`:

```
# deployment/kustomization.yaml
resources:
  - ../../base          # only if the base is in the SAME repo
images:
  - name: myapp
    newTag: "1.4.2"
```

ONE REPO PER ENVIRONMENT LIMITS THE USUAL OVERLAY PATTERN

The classic `base/` + `overlays/{dev,test,prod}` layout assumes all environments live in one repo; here they do not. Either keep a `base/` inside each deployment repo, or use Kustomize purely for its transformers — `images:`, `configMapGenerator`, `patches` — against local resources. ArgoCD cannot reach a base in a different repo without a multi-source Application.

`images:` is a clean way to do releases: the tag lives in one place and the bump is a one-line, reviewable diff.

App-of-apps

One Application whose job is to create other Applications. Useful when what the platform onboarded as a single "app" is really several deployable components — an API, a worker, a migration job — that you want to sync, health-check and roll back independently.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: mytenant-myapp-api
  namespace: ns-mytenant-myapp      # your namespace, not argocd
spec:
  project: tenant-mytenant          # must be your tenant's project
  source:
    repoURL: git@ssh.dev.azure.com:v3/capiomks/CMKS-MyTenant-MyApp/PROD
    targetRevision: main
    path: components/api
  destination:
    server: https://kubernetes.default.svc
    namespace: ns-mytenant-myapp
  syncPolicy:
    automated: { prune: true, selfHeal: true }

```

This works because the platform enables it explicitly: ArgoCD runs with `application.namespaces: "*" ,` and every tenant namespace carries the label `argocd.argoproj.io/managed-by: argocd ,` so Application resources are picked up from your own namespace.

- `project:` must be your tenant's AppProject. Another tenant's project is rejected.
- `repoURL` must be on your tenant's allowlist.
- Child Applications inherit every AppProject restriction — an app-of-apps grants no extra privilege.
- Deleting the parent prunes the children. Be deliberate about it in prod.

ApplicationSets

Generates Applications from a template. Worth it when you have many near-identical deployments — per-country, per-region or per-customer instances of the same service.

SCM PROVIDERS ARE DISABLED

The ApplicationSet controller runs with `--enable-scm-providers=false ,` so the **SCM Provider** and **Pull Request** generators do not work — you cannot generate an Application per Azure DevOps repo or per open PR. The **List**, **Git** (files and directories), **Cluster**, **Matrix** and **Merge** generators all work.

Which should you pick?

- **One service, three environments** → plain manifests. Do not over-engineer.
- **Many near-identical values per environment** → Helm with a committed values file.
- **You only vary the image tag and a few fields** → Kustomize with `images: .`
- **Several components under one onboarded app** → app-of-apps.
- **Dozens of generated instances** → ApplicationSet with a List or Git generator.

Whatever you choose, the [guardrails](#) apply identically. A Helm chart is not a way around Kyverno.

Inbound traffic

There are exactly two ways in, and you don't choose freely — the `facing` field in your config decides:

GATEWAY	LOAD BALANCER	REACHABLE FROM
<code>internal</code>	OpenStack internal LB	Capio network / VPN — staff and service-to-service
<code>external</code>	Public LB	The open internet — patient-facing

The platform creates the **Gateway listener** and its **TLS certificate**. You create the **HTTPRoute** that binds your Service to that listener:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: bookingapi
  namespace: ns-mytenant-bookingapi
spec:
  parentRefs:
    - name: internal          # or "external"
      namespace: internal-ingress # or "external-ingress"
      sectionName: mytenant-bookingapi # the listener
  hostnames:
    - bookingapi.prod01.capio.eu
  rules:
    - backendRefs:
        - { name: bookingapi, port: 8080 }
```

Multiple hostnames get one listener each: the first uses the namespace-derived `sectionName`, later ones are suffixed `-1`, `-2`. Write one HTTPRoute per hostname.

■ YOU CANNOT CREATE GATEWAY

■ YOU CANNOT CREATE CERTIFICATE

Both are blocked by the ArgoCD AppProject. TLS is issued and renewed automatically — Let's Encrypt via Azure DNS-01 for public names, the internal CA for in-cluster names.

Outbound traffic

Every app namespace gets a default egress NetworkPolicy. It draws the line in one place only: **inside the cluster versus outside it**.

WHERE YOU ARE GOING	WHAT IS ALLOWED
Anything inside the cluster — any pod, in any namespace	All ports. No restriction.
Anything outside the cluster , addressed by IP	80, 443, 53, 8053, 5432, 5433, 5671, 5672

This is worth reading twice, because it is the opposite of what most people assume.

In-cluster traffic is not port-restricted. The platform's PostgreSQL, RabbitMQ and Redis are reachable on whatever port they listen on — not because those ports are individually allowlisted, but because egress to every namespace in the cluster is open. The same applies to another team's service in another namespace. If you can resolve it in-cluster, egress policy will not stop you reaching it.

Leaving the cluster is where the restriction bites. Traffic to an external IP is limited to that port list. The 5432 and 5671 entries there are for reaching a PostgreSQL or RabbitMQ that lives *outside* the cluster — they are not what makes the platform's own instances work.

SILENT FAILURE MODE

Anything else leaving the cluster is blocked, and it fails as a **connection timeout, not a clean refusal**. SQL Server on 1433, LDAP on 389/636, SMB on 445, Oracle on 1521, a bespoke TCP protocol — each needs an egress rule added for your namespace by the platform team.

To request one, give them:

GIVE THEM	WHY
Destination hostname and IP/CIDR	rules are written against addresses
Port and protocol	e.g. TCP 1433
Environments	each one is a separate rule
What it is for	one line; it ends up in the change record

Know your destination hosts and ports early. This is an infrastructure change with a review and an apply behind it, not a self-service toggle — and if the destination is a **Capio internal system**, it needs considerably more than this rule. See the next section.

Reaching Capio internal systems

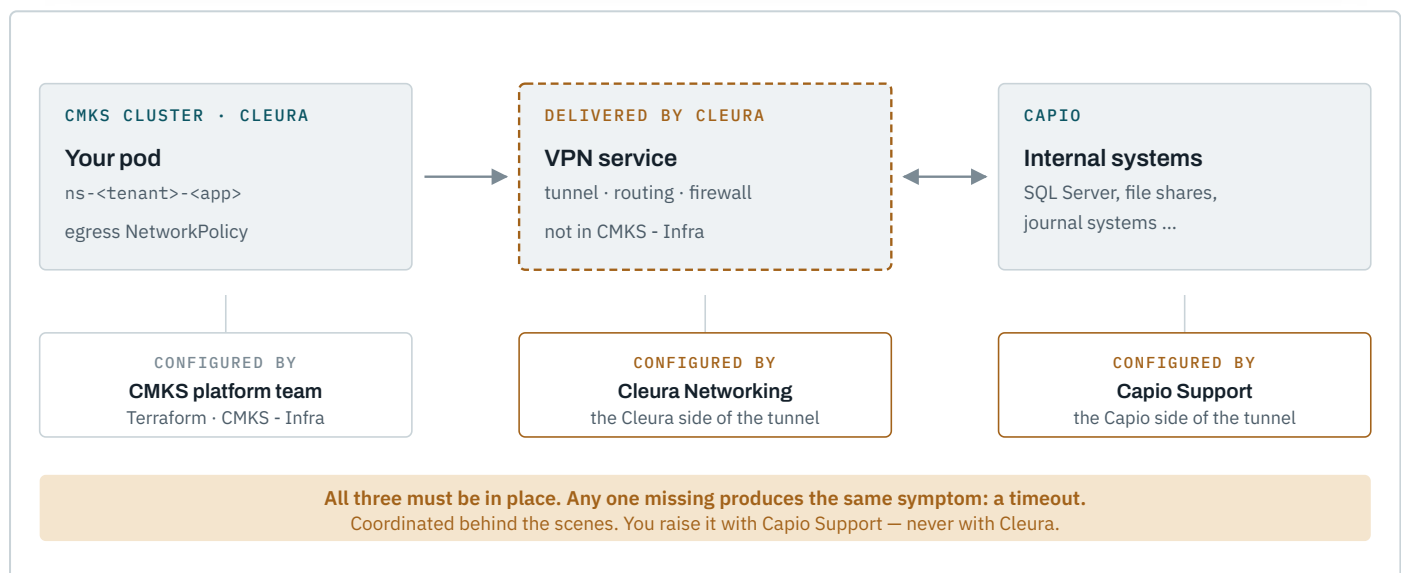
■ RAISE IT WITH CAPIO SUPPORT

This is the part that surprises people, and the part with the longest lead time.

CMKS clusters run at **Cleura**. Capio's internal systems do not. The link between them is a **VPN service delivered by Cleura** — it is not part of the CMKS infrastructure code, and neither you nor the CMKS platform team can change it alone.

YOU RAISE EVERYTHING WITH CAPIO SUPPORT

You will never contact Cleura, and you do not need to work out which party owns which piece. Capio Support is your single entry point; they coordinate the rest. The diagram below is here so you understand *why* it takes as long as it does — not so you can chase people.



Three segments, three owners, three change processes — none of them self-service.

What this means in practice

- **Capio Support is your only contact.** Whatever the piece, whoever ends up doing the work, the request goes to Capio Support and they coordinate with the Capio Networking team and — where Cleura's side is involved — with Cleura. Developers do not raise anything with Cleura directly.
- **Two organisations, two change processes.** That is the real reason for the lead time, and it is why nobody can simply toggle this for you.
- **Three changes have to line up** before traffic flows: the Cleura-side VPN and routing config, the Capio-side firewall and routing config, and the egress NetworkPolicy for your namespace.
- **Each environment is its own request.** A route that works from `dev02` tells you nothing about `prod01`.

- **Lead time is measured in weeks, not hours.** Raise it when you start designing the integration, not when you're ready to test it.

What to bring to Capio Support

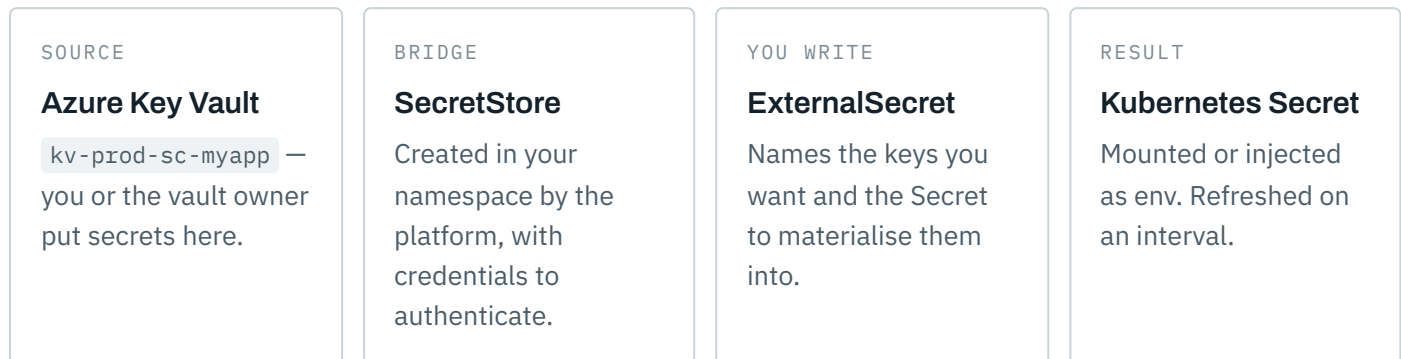
ITEM	WHY IT'S NEEDED
Source: cluster + namespace	identifies your egress side
Destination: FQDN and IP/CIDR	firewall rules are written against IPs
Port and protocol	e.g. TCP 1433
Direction	almost always outbound; inbound needs a much stronger case
Environment(s)	dev / test / prod are separate approvals
Data classification and purpose	patient data crossing a boundary attracts review

DESIGN ADVICE

Every VPN dependency couples your release schedule to two networking teams. Where you have the choice, prefer an **HTTPS API on 443** over a direct database or file-share connection: 443 egress already works out of the box, and an API is far easier to get approved than SQL access across the boundary.

Secrets

Secrets never live in Git and never live in your manifests. They live in **Azure Key Vault** — one vault per app per environment — and are pulled into your namespace by the **External Secrets Operator**.



```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: myapp-secrets
  namespace: ns-mytenant-bookingapi
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: azurekv-mytenant-bookingapi # platform-created
    kind: SecretStore
  target:
    name: myapp-secrets
  data:
    - secretKey: ApiKey
      remoteRef:
        key: payment-provider-api-key # the name in Key Vault
```

To add a secret: put it in Key Vault, then reference it from your `ExternalSecret`. No platform change required.

Platform services you can use

These run once per environment and are shared. You ask for them during onboarding or later; the platform provisions them and the credentials arrive as a **Kubernetes Secret** in **your namespace**, ready to mount or inject as environment variables.

SERVICE	WHAT YOU GET	WHAT TO ASK FOR
PostgreSQL CloudNativePG	One or more databases in the shared cluster, each with its own owner role, backed up automatically	The database names you need, per environment
RabbitMQ	Queues, exchanges and bindings, reached over AMQP — there is no management UI access	Queue and exchange names, fanout or not, whether external consumers need access, message TTLs
Redis	A managed Redis instance	That you need Redis, and roughly how much you will store
Airflow	DAGs synced from a Git repo	Relevant for data workloads — talk to the platform team about fit

Your database Secret carries `host` , `username` , `password` and two connection URIs: one read-write, one pointing at the **read-only replica**. Use the read-only one for reporting and read-heavy paths.

AVAILABILITY DIFFERS PER ENVIRONMENT

Not every platform service is enabled everywhere, and the list above is what you can normally expect. Confirm with the platform team before you design around a service — especially if you need it in production.

Always reach these over **internal cluster DNS**, never a public hostname. Traffic that leaves and re-enters the cluster is slower, crosses the gateway unnecessarily, and may be blocked by your egress policy.

Guardrails

Enforced by the **ArgoCD AppProject** (blocked at sync) and by **Kyverno** (blocked at admission). These are not suggestions — a violating manifest will not become a running pod.

YOU CANNOT	ENFORCED BY	WHY
Create any cluster-scoped resource — <code>ClusterRole</code> , <code>ClusterRoleBinding</code> , CRDs	AppProject	tenant isolation
Create <code>Namespace</code>	AppProject	namespaces ship with quota and policy
Create or edit <code>ResourceQuota</code> / <code>LimitRange</code>	AppProject	capacity is allocated, not self-served
Create <code>Gateway</code> — Gateway API or Istio	AppProject	ingress and TLS are platform-owned
Create <code>Certificate</code>	AppProject	issued automatically by cert-manager
Create <code>LoadBalancer</code> or <code>NodePort</code> Services	Kyverno	all ingress goes through shared gateways
Run without <code>seccompProfile: RuntimeDefault</code>	Kyverno · Enforce	baseline hardening
Run with a writable root filesystem	Kyverno · Enforce	baseline hardening
Auto-mount the service account token	Kyverno	least privilege
Use a Git repo that is not on your tenant's allowlist	AppProject <code>sourceRepos</code>	tenant isolation

Also in force:

- **Pod Security Admission** at `baseline` — enforce, audit and warn — on every tenant namespace.
- **Images must come from the ACR provisioned for your app.** That registry is created alongside your app config; your build pushes there and your manifests pull from there. Registry restrictions are enforced by policy — build against your own ACR from the start.
- **ResourceQuota** per namespace. Exceed it and pods stop scheduling — with a quota error, not a crash loop.
- **Istio sidecar injection** with mTLS between meshed workloads.

Life without kubectl

WHAT YOU'D HAVE DONE	WHAT YOU DO INSTEAD
kubectl get pods	ArgoCD — your Application's resource tree, live sync and health status
kubectl logs	ArgoCD pod logs — your project role has explicit <code>logs</code> , <code>get</code> permission
kubectl describe	ArgoCD resource view — events and conditions
kubectl apply	commit to the deployment repo; ArgoCD syncs
kubectl rollout restart	ArgoCD Sync / Refresh , or commit a change
kubectl rollout undo	revert the commit — ArgoCD rolls forward to the previous state
kubectl top	Grafana — Kubernetes and namespace dashboards
kubectl exec	not available — build in health endpoints, structured logs and metrics
kubectl port-forward	not available — use the internal hostname from a VPN-connected client

Getting access — VPN, Jumphost, ArgoCD

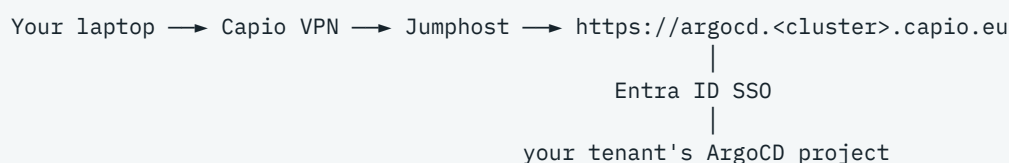
■ CAPIO SUPPORT

■ CMKS PLATFORM TEAM

Sort this out before your first deployment, not after. Because there is no `kubectl`, ArgoCD is your only window into the cluster. A developer who can push to the deployment repo but cannot open ArgoCD has no way to see whether the sync worked, read a pod log, or tell a crash loop from a failed image pull.

None of the platform UIs are on the public internet — they are all published on the **internal** gateway. Reaching them takes three separate things, granted by three different routes:

#	WHAT YOU NEED	HOW YOU GET IT
1	Connected to the Cpio VPN	Your normal Cpio client access — Cpio Support
2	Access to the Jumphost machine	The platform UIs are reached <i>through</i> the Jumphost, not directly from your laptop
3	ArgoCD access for that environment	Your tenant's Entra ID group must be registered for that environment's ArgoCD



ACCESS IS PER ENVIRONMENT

Being able to open dev's ArgoCD tells you nothing about test or prod — each environment has its own ArgoCD instance and its own group registration. Request every environment you are expected to support, and verify each one by actually logging in. Discovering on release day that nobody on the team can open prod's ArgoCD is a common and entirely avoidable delay.

Ask the platform team for the current Jumphost details and for ArgoCD registration; ask Capio Support for VPN and Jumphost access itself.

Your tools

All of these are reached the same way — VPN, then Jumphost — and all are SSO-protected with **Microsoft Entra ID**:

TOOL	URL	FOR
ArgoCD	<code>argocd.<cluster>.capio.eu</code>	sync status, resource tree, pod logs, manual sync
Grafana	<code>grafana.<cluster>.capio.eu</code>	metrics, resource usage, cost per namespace

`<cluster>` is `dev02`, `test02` or `prod01` — each environment has its own instance of each tool, and you log in to the one for the environment you are working on.

That is the whole list. In particular there is **no RabbitMQ management UI** and **no service-mesh UI** for developers: RabbitMQ is available to your application over **AMQP only**. If you need to know how deep a queue is or how fast it is draining, your application has to expose that as a metric — you cannot go and look.

What you can do once you are in

Your tenant's ArgoCD project grants one of two roles:

- **dev** — view applications, view logs, trigger a sync. No delete, no create.
- **admin** — full control of applications and ApplicationSets within your project.

`exec` is not granted at any level, in any environment, to anyone.

CONSEQUENCE FOR YOUR APP

Because there is no `exec` and no `port-forward`, **observability is not optional**. Ship a real health endpoint, structured JSON logs to stdout, and Prometheus metrics. Skip this and you will be debugging blind — and the one tool you do have is several access hops away.

Checklist

Before you write any Kubernetes YAML

- ☐ Tenant identified; you're in the right Entra ID group
- ☒ Cpio VPN access working
- ☒ Jump host access working
- ☒ ArgoCD reachable and you can log in — verified separately for each environment
- ☐ Onboarding form submitted for the tenant and app
- ☐ Hostname(s) agreed, and `internal` vs `external` decided
- ☐ Key Vault populated for each environment
- ☐ CMKS project and its `DEV` / `TEST` / `PROD` repos handed over
- ☐ Your app's ACR handed over, and your build pipeline pushes to it
- ☒ Every outbound dependency listed — host, IP, port, protocol
- ☒ VPN / connectivity requests raised with Cpio Support, per environment
- ☐ Databases, queues or Redis requested
- ☐ Resource footprint agreed — CPU, memory, pod count

In the deployment repo

- ☐ `deployment/` exists on the configured branch of the deployment repo
- ☐ Image pulled from your app's ACR by an immutable tag or digest — never `:latest`
- ☐ `seccompProfile: RuntimeDefault` set
- ☐ `readOnlyRootFilesystem: true` set, with `emptyDir` mounts for every writable path
- ☐ `automountServiceAccountToken: false`
- ☐ Resource requests *and* limits on every container
- ☐ Service is `ClusterIP`
- ☐ HTTPRoute references the right gateway, namespace and `sectionName`
- ☐ `ExternalSecret` references the platform-created `SecretStore`
- ☐ No secrets, no `Gateway`, no `Certificate`, no cluster-scoped resources in Git
- ☐ Health endpoints, structured stdout logging, metrics endpoint

Verify

- ☐ ArgoCD shows the Application **Synced** and **Healthy**
- ☐ Hostname resolves and serves HTTPS with a valid certificate
- ☐ Outbound dependencies actually connect — a timeout means a missing NetworkPolicy or VPN rule
- ☐ Metrics and logs visible in Grafana

Who to ask

There are only two places you ever send a request.

TOPIC	WHO
A new tenant or a new app	the onboarding form
Namespace, quota, ArgoCD project, hostname, TLS, databases, queues, egress policy	CMKS platform team
ArgoCD registration for your tenant, per environment	CMKS platform team
Capio VPN access, Jumphost access	Capio Support
Connectivity to Capio internal systems — firewall rules, routing, anything VPN	Capio Support
Key Vault contents, Entra ID group membership	your own team / Capio IT

YOU NEVER CONTACT CLEURA

Cleura operates the underlying cloud and their side of the VPN, but they are not a party you raise anything with. Everything that touches Cleura goes through **Capio Support**, who coordinate it.

This guide is deliberately standalone — everything a developer needs, and nothing more. The platform team maintains deeper operator-facing documentation in the **CMKS - Infra** repository: architecture overview, networking and ingress, tenant model, data services, security and compliance, and observability. Those assume access to the infrastructure repository — ask the platform team if you need something from them.